

Práctica: Filtrado, Morfología y Detección de Bordes

Objetivo

Aplicar técnicas de filtrado y morfología para reducir ruido, mejorar imágenes y detectar bordes utilizando OpenCV.

Introducción al filtrado de imágenes – Filtros pasa baja

Los filtros pasa baja suavizan la imagen eliminando detalles pequeños y ruido.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('imagen.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

suave = cv2.blur(img, (15,15))

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img)
plt.title('Original')

plt.subplot(1,2,2)
plt.imshow(suave)
plt.title('Filtro pasa baja')

plt.show()
```

Actividad: Cambiar el tamaño del filtro: (3,3), (7,7), (15,15) y explicar qué ocurre mientras aumenta.

Blurring

```
blur = cv2.blur(img, (9,9))

plt.imshow(blur)
plt.title('Blurring')
plt.axis('off')
plt.show()
```

Preguntas: ¿Qué detalles desaparecen? Y ¿Cuándo sería útil este filtro?

Desenfoque Gaussiano

```
gauss = cv2.GaussianBlur(img, (9,9), 0)

plt.imshow(gauss)
plt.title('Gaussian Blur')
plt.axis('off')
plt.show()
```

Comparación:

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img)
plt.title('Original')

plt.subplot(1,3,2)
plt.imshow(blur)
plt.title('Blur')

plt.subplot(1,3,3)
plt.imshow(gauss)
plt.title('Gaussian Blur')

plt.show()
```

Actividad: Determinar cuál conserva mejor los bordes.

Desenfoque medio

```
median = cv2.medianBlur(img, 9)

plt.imshow(median)
plt.title('Median Blur')
plt.axis('off')
plt.show()
```

Explicar: Es especialmente útil para ruido sal y pimienta.

Actividad: Aplicar el filtro a una imagen con ruido.

Filtrado bilateral

El filtro bilateral suaviza la imagen pero mantiene los bordes.

```
bilateral = cv2.bilateralFilter(img, 9, 75, 75)

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.imshow(img)
plt.title('Original')

plt.subplot(1,2,2)
plt.imshow(bilateral)
plt.title('Bilateral')

plt.show()
```

Actividad: Comparar con Gaussian Blur, y explicar por qué el bilateral conserva mejor los contornos.

Restauración de una imagen antigua

```
img = cv2.imread('imagen_antigua.jpg', cv2.IMREAD_GRAYSCALE)

restaurada = cv2.medianBlur(img, 5)
restaurada = cv2.equalizeHist(restaurada)

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title('Antigua')

plt.subplot(1,2,2)
plt.imshow(restaurada, cmap='gray')
plt.title('Restaurada')

plt.show()
```

Actividad: Buscar una fotografía antigua y mejorarla.

Transformaciones morfológicas

Las transformaciones morfológicas trabajan sobre imágenes binarias.

```
img = cv2.imread('binaria.png', cv2.IMREAD_GRAYSCALE)

kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
```

Erosión y dilatación para cierre de formas

```
erosion = cv2.erode(img, kernel, iterations=1)
dilatacion = cv2.dilate(erosion, kernel, iterations=1)

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title('Original')

plt.subplot(1,3,2)
plt.imshow(erosion, cmap='gray')
plt.title('Erosion')

plt.subplot(1,3,3)
plt.imshow(dilatacion, cmap='gray')
plt.title('Dilatacion')

plt.show()
```

Actividad: Observar cómo se cierran pequeños huecos.

Morfología de cierre

```
cierre = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)

plt.imshow(cierre, cmap='gray')
plt.title('Cierre')
plt.axis('off')
plt.show()
```

Explicar:

- Cierre = dilatación + erosión
- Sirve para unir regiones separadas.

Erosión y dilatación para eliminar elementos no deseados

```
erosion = cv2.erode(img, kernel, iterations=2)
```

Actividad: Eliminar pequeños puntos blancos o ruido.

Morfología de apertura

```
apertura = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)

plt.imshow(apertura, cmap='gray')
plt.title('Apertura')
plt.axis('off')
plt.show()
```

Explicar:

- Apertura = erosión + dilatación
- Elimina pequeños objetos.

Filtro pasa alta – Sobel

El filtro Sobel detecta cambios bruscos, es decir, bordes.

```
img = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)

sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.imshow(sobelx, cmap='gray')
plt.title('Sobel X')

plt.subplot(1,2,2)
plt.imshow(sobely, cmap='gray')
plt.title('Sobel Y')

plt.show()
```

Actividad: Explicar por qué Sobel X detecta bordes verticales y Sobel Y horizontales.

Aplicando el filtro Sobel

```
sobel = cv2.magnitude(sobelx, sobely)

plt.imshow(sobel, cmap='gray')
plt.title('Magnitud Sobel')
plt.axis('off')
plt.show()
```

Actividad: Comparar con la imagen original y identificar qué objetos se distinguen mejor.

Filtro Canny y coloración de bordes

```
img_color = cv2.imread('imagen.jpg')
gris = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)

canny = cv2.Canny(gris, 100, 200)

img_rgb = cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB)
img_rgb[canny > 0] = [255,0,0]

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.imshow(canny, cmap='gray')
plt.title('Canny')

plt.subplot(1,2,2)
plt.imshow(img_rgb)
plt.title('Bordes coloreados')

plt.show()
```

Actividad: Probar distintos valores en `cv2.Canny(gris, umbral1, umbral2)` y comparar con Sobel.